THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 03 OF 04

# System Design #10 — Change Data Capture (CDC) Pipeline

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

03



YOUR STUDY COMPANION

**Solution Architect interview preparation I Reference architecture and decision guide Edition:** 3 November 2026 **Example stack:** PostgreSQL logical decoding + Debezium + Kafka + independent sinks **Scope:** Generic financial-services scenario; this is not a description of American Express's internal systems.

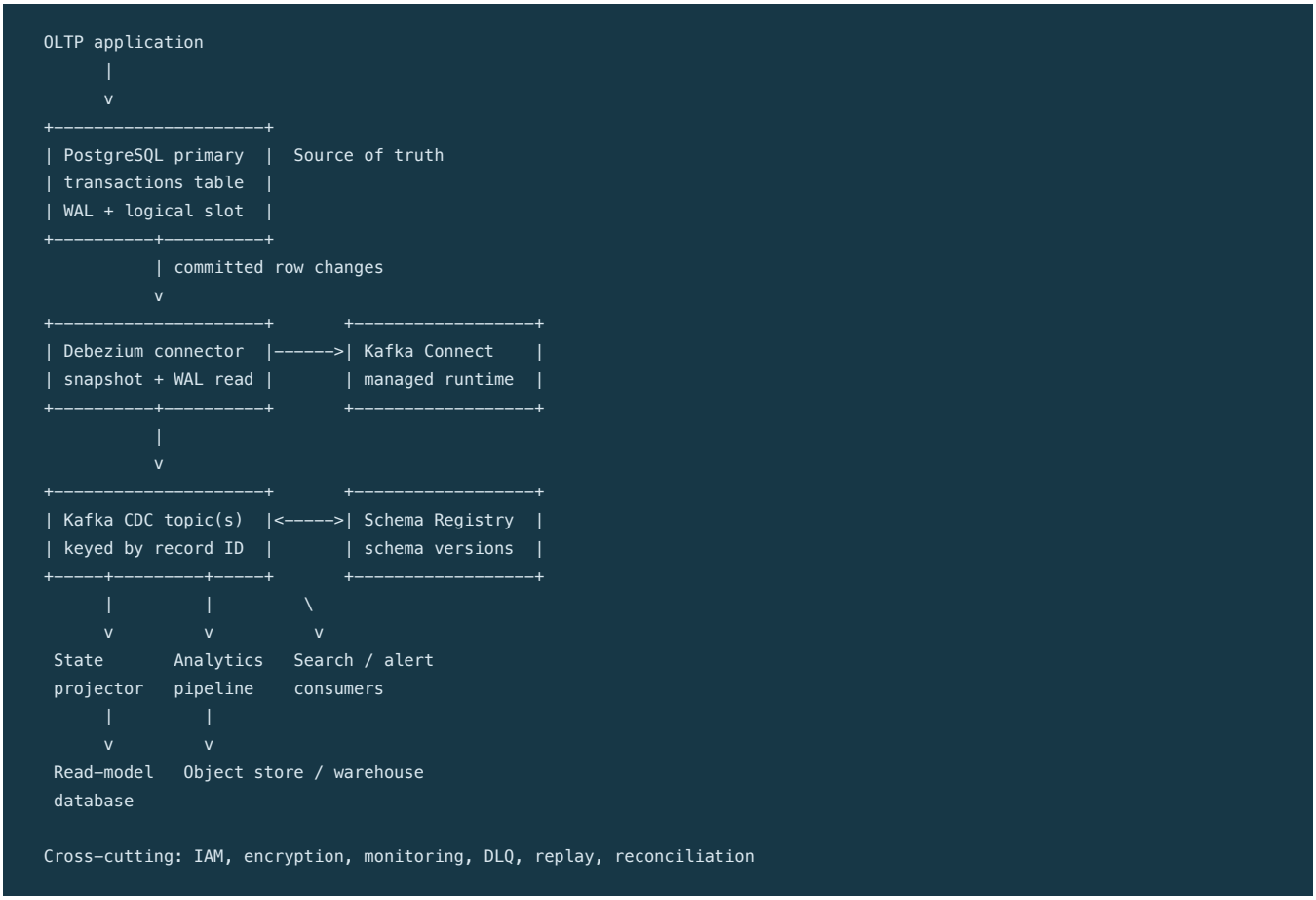
## Executive summary

**Problem.** A financial services application writes transaction-status records to a PostgreSQL OLTP database. Search, customer-service dashboards, an operational data store, and analytics all need fresh changes. Repeatedly querying the production database would increase load, couple consumers to the source schema, and still make deletions and missed updates difficult to handle.

**Recommended design.** Capture committed row changes from PostgreSQL's write-ahead log (WAL) using a Debezium source connector running in Kafka Connect. Store the resulting change events in partitioned Kafka topics with an explicit schema contract. Independent consumers build projections and analytical datasets. Use snapshot-plus-streaming for bootstrap, at-least-once processing with idempotent sinks, replayable retention, end-to-end monitoring, and strict sensitive-data controls.

**Core trade-off.** CDC provides low-source-impact, near-real-time replication, but it is *not* the same as an atomic business-event bus. It introduces eventual consistency, schema coupling, source WAL-retention risk, and replay/duplicate handling. Use a transactional outbox when consumers need semantically meaningful domain events rather than row-level mutations.

### Architecture at a glance




NAVIGATION

# Inside this guide


01 1. Define the problem and boundaries

02 2. Derive the architectural choices

03 3. Component responsibilities

04 4. End-to-end request and data flows

05 5. Event contract and data semantics

06 6. Consistency and delivery guarantees

07 7. Partitioning, scaling, and retention

08 8. Failure modes and recovery strategy

09 9. Security and compliance design

10 10. Monitoring, SLOs, and operational readiness

11 11. Validation strategy before production

12 12. Alternatives and when to select them

13 13. Architecture decision records — concise summary

14 14. Interview-ready explanation (one-minute narrative)

15 15. Official technical references

CHAPTER

# 1. Define the problem and boundaries

## 1.1 Functional requirements

1. Capture **INSERT**, **UPDATE**, and **DELETE** operations for an approved set of source tables after they commit.
2. Deliver new changes to multiple independent downstream applications with low delay.
3. Bootstrap an empty downstream system with existing rows **without a gap** between initial load and live changes.
4. Allow a consumer to restart, catch up, or replay retained changes without requesting them again from the source.
5. Support schema evolution, explicit deletion handling, and controlled downstream access.
6. Detect missing, stuck, duplicated, delayed, or malformed processing; provide repair procedures.

## 1.2 Non-functional requirements — proposed interview assumptions

These numbers are **illustrative design targets, not verified production requirements**.

| Dimension | Assumed target | Architectural consequence |
|---|---|---|
| Freshness | 99% of healthy-path updates visible downstream within 10 s | Streaming CDC, end-to-end lag metric |
| Availability | Source database must keep serving core transactions if analytics is down | Decouple source and sinks using broker |
| Correctness | No silently lost committed source changes inside supported retention/recovery window | Durable offsets, WAL monitoring, replay, reconciliation |
| Delivery | At-least-once end-to-end; duplicates tolerated and suppressed at sinks | Idempotent application of changes |
| Recovery | Rebuild a sink from retained stream or a fresh snapshot | Topic retention + backfill runbook |
| Ordering | Preserve order for the same business key in each topic | Stable record key and Kafka partitioning |
| Privacy | Do not replicate raw card data or unnecessary personal data | Source allowlist and field-level controls |

**Out of scope.** This pipeline does not authorize payments, decide financial truth, guarantee synchronous updates across services, or replace the source ledger. The source OLTP system remains authoritative.

## 1.3 Scale estimate (assumptions, not vendor limits)

| Input | Assumption / calculation |
|---|---|
| Average source change rate | 500 events/s |
| Peak rate | 5,000 events/s (10x average) |
| Average serialized record | 1 KB after envelope/metadata, for initial sizing |
| Events per day | 500 × 86,400 = **43.2 million** |
| Logical data per day | About **43.2 GB/day** (decimal units) |
| Seven-day logical retention | About **302.4 GB** |
| Three replicas, before overhead | About **907 GB** of broker storage |

**Why estimate?** The dominant decisions depend on change rate, peak rate, record size, number of sinks, snapshot size, and retention. Recalculate for compression, replication, indexes, log compaction, safety headroom, and expected growth. The peak rate determines throughput capacity; average rate drives retained volume.

CHAPTER

# 2. Derive the architectural choices

| Decision | Select | Why / when to reconsider |
|---|---|---|
| Capture mechanism | **Log-based CDC** from PostgreSQL WAL | Reads committed changes and avoids repeated table scans. Polling is simpler for low-volume, timestamped tables but can miss deletes or short-lived states. |
| Capture engine | **Debezium PostgreSQL connector** | Provides snapshot and WAL streaming, documented operations, and Kafka Connect integration. Check supported versions/operations. |
| Integration layer | **Kafka** | Durable buffer, multiple consumer groups, retained replay. A queue may suffice if only one worker consumes each task and replay is unnecessary. |
| Connector runtime | **Kafka Connect distributed workers** | Externalized connector lifecycle and offsets. Important: the PostgreSQL connector itself runs **one source task**, so more workers do not parallelize a single PostgreSQL capture task. |
| Topic strategy | **Topic per governed source table** initially | Clear ownership/schema and table-level access. Aggregate across tables only with explicit ordering/contract requirements. |
| Partition key | **Stable primary/business record ID** | All events for a key normally land in one partition and can be applied in key order. A hot key may need special treatment. |
| Serialization | **Versioned Avro/Protobuf/JSON Schema**, with a registry where supported | Makes breaking changes visible before they disrupt consumers. Contract evolution must be governed. |
| Sink semantics | **Idempotent upsert/delete + offset discipline** | Replay and connector recovery can produce duplicates. Do not assume one-and-only-one side effect. |
| Analytical storage | **Object store / warehouse** | Cheap historical storage and analytical queries; not the synchronous source of truth. |
| Business events | **Transactional outbox (conditional)** | Choose when consumers need `PaymentAuthorized` rather than a mutable row's `status` update. |

## 2.1 Why not database polling?

Polling `WHERE updated_at > last_poll` seems easy, but clock precision, transaction timing, concurrent updates, and deletes make it fragile without additional controls. Continuous scans also burden the OLTP database. Prefer polling only when scale and consistency needs are modest and the source cannot expose logs.

## 2.2 Why Kafka rather than direct Debezium-to-warehouse?

A direct sink is simpler for one destination. Kafka adds operational complexity, but it **decouples consumer availability**, supports independent consumer groups, and retains a replayable change stream. We select it because several downstream systems need the same updates with

different speeds and recovery needs.

CHAPTER

# 3. Component responsibilities

| Component | Responsibility | Critical design concern |
|---|---|---|
| PostgreSQL OLTP | Commit source-of-truth writes | Keep transaction workload healthy; support logical decoding |
| Publication / replication slot | Define captured tables and track WAL position | Lagging slot retains WAL and can exhaust disk |
| Debezium source connector | Perform consistent initial snapshot, then stream change events | Source task state, connector offsets, failure recovery |
| Kafka Connect workers | Host and supervise connector tasks | HA for workers is not source parallelism |
| Kafka brokers and topics | Buffer, partition, retain, replay records | Capacity, replication, retention, ACLs |
| Schema Registry | Publish and check schema versions | Compatibility across producers and consumers |
| State projector | Apply latest valid record state to read model | Idempotency, deletes, stale event protection |
| Analytics pipeline | Validate/transform and load history or aggregates | Replay, dedupe, late arrivals, batch commits |
| Observability + operations | Lag, slot health, DLQs, drift detection, recovery | Detect failure before WAL or freshness budgets are exceeded |

CHAPTER

# 4. End-to-end request and data flows

## 4.1 One committed transaction update

1. Application updates a transaction-status row inside a normal database transaction.
2. PostgreSQL commits it and writes the change to WAL; **no direct application-to-Kafka dual write** is required.
3. The replication slot exposes committed changes to Debezium's PostgreSQL connector.
4. Debezium creates a change event containing the operation (`c`, `u`, `d`, or snapshot `r`), key, row state, and source metadata.
5. Kafka receives the record in the corresponding topic; the record key identifies the transaction row.
6. Consumers read independently. The read-model consumer upserts a row or deletes it as appropriate; the analytics consumer appends to a governed historical dataset.
7. A consumer records processing progress only after its durable sink update succeeds (or coordinates both atomically when available).

## 4.2 Initial snapshot without a gap

A brand-new Kafka topic cannot reconstruct rows that changed **before** CDC was enabled merely by replaying current WAL. Debezium's typical initial mode captures a **consistent snapshot** and then streams subsequent changes from the appropriate log position.

```
Time ------------------------------------------------------>
   | snapshot start          | snapshot finish
   |-------------------------|
           source writes keep occurring
```

```
    [read existing rows]       [consume WAL after snapshot point]
                \             /
                  coherent downstream state
```

**Why not `SELECT *` followed by `start streaming`?** Writes committed between the two operations can be missed. Snapshot/stream coordination is part of the CDC connector's job. Large initial snapshots need capacity planning and may increase source read/IO load. Incremental snapshots can backfill selected tables in chunks when appropriate.

### 4.3 Retry and replay

If a consumer fails after updating its sink but before committing its offset, the same Kafka record may be processed again. Therefore a sink must handle repeated upserts/deletes, or dedupe by stable event identity and guard against stale ordering. If a consumer's retained offsets are gone, replay requires a controlled new snapshot or historical archive, not wishful recovery.

---

CHAPTER

## 5. Event contract and data semantics

### 5.1 Illustrative record — simplified, not a literal full Debezium payload

```
{
  "key": { "transaction_id": "txn_1032" },
  "value": {
    "op": "u",
    "before": { "status": "PENDING" },
    "after": {
      "transaction_id": "txn_1032",
      "status": "APPROVED",
      "amount_minor": 2500,
      "currency": "USD"
    },
    "source": { "table": "transactions", "lsn": "source-position" },
    "event_time": "2026-11-03T16:00:00Z"
  }
}
```

The concrete Debezium envelope includes more fields than this example and depends on connector configuration. Never treat the example's `lsn` string as a guaranteed globally unique event ID. Use schema-provided source metadata and business identifiers correctly.

### 5.2 The essential semantics

- **Snapshot `r`:** represents existing row state during a snapshot; **create `c`, update `u`, delete `d`** represent subsequent changes.
- **Deletes:** interpret connector delete events and optional tombstones deliberately. A tombstone is useful for Kafka log compaction, but it is not by itself a universal sink-delete command.
- **Ordering:** Kafka preserves order **inside a partition**. Using a stable row key gives practical per-key ordering in a topic. There is no automatic total order across topics or partitions.
- **Transaction boundaries:** multi-table source transactions may produce several records/topics. If consumers require transaction-level atomic application, enable/use transaction metadata or design an explicit downstream coordination mechanism.
- **Schemas:** use additive changes where possible; test rename, type changes, primary-key changes, and nullability. Schema compatibility rules vary with selected format and policy.
- **PII:** exclude or pseudonymize protected fields **before exposing them to broadly shared topics**. Kafka retention, logs, backups, and replays extend the data's exposure lifetime.

### 5.3 Idempotent sink pattern

For a latest-state projector: (a) consume by stable key, (b) apply upsert/delete in a database transaction, and (c) commit/checkpoint progress only after the write. If the sink uses a separate offset store, protect against replayed or older events using a stable source sequence or transactional offset/state record. **Do not rely on a timestamp alone** to determine correct row order.

For financial side effects (e.g., crediting an account), **do not trigger a non-idempotent money movement directly from a raw row-change event**. Prefer a transactional outbox event with a durable business event ID and a consumer-side uniqueness constraint.

---

CHAPTER

## 6. Consistency and delivery guarantees

**Source correctness:** PostgreSQL is authoritative. CDC normally observes committed data, but the stream can lag the source.

**Delivery guarantee:** Design the end-to-end system as **at least once**, with idempotent sinks. Kafka Connect supports exactly-once source connector capabilities in supported configurations, but this does **not** automatically make external sink side effects exactly once. Debezium documents duplicates after some failures.

**Ordering contract:** Promise only per-key order within a topic partition, and specify how keys and partition counts are managed. Changing partition counts can change key routing, so plan such changes carefully.

**Cross-table consistency:** If a business operation updates `transactions` and `balances`, the two Kafka topics are not a globally atomic read view. Downstream consumers needing a consistent financial view should use an authoritative ledger or transactional projection strategy; CDC itself does not invent that guarantee.

**Eventual consistency:** The read model can temporarily disagree with the source during healthy operation. Define a measured freshness SLO and a clear response when it is exceeded.

---

CHAPTER

## 7. Partitioning, scaling, and retention

### 7.1 How scaling actually works

- Capture throughput is bounded by database WAL generation, logical decoding, network, and Debezium's **single PostgreSQL source task**.
- Kafka can parallelize consumers using partitions and consumer-group members. Choose enough partitions for expected sink throughput, while considering operational cost.
- Give different downstream systems **different consumer groups**. Two services in the same group share work rather than each getting the full stream.
- Scale hot or expensive sinks independently. Prevent one failed sink from blocking unrelated consumers.
- Treat large snapshots separately from steady-state throughput; throttle or schedule when necessary to protect OLTP.

### 7.2 Retention and replay

- **Kafka retention is finite.** Choose a retention window long enough for normal outages and reprocessing. Archive changes to object storage if reprocessing must span a longer period.
- **PostgreSQL WAL retention is a different budget.** If Debezium stalls, a logical replication slot can pin WAL until the source disk fills. Alert and recover well before that happens.
- Topic **log compaction** may be appropriate for latest-state reconstruction, while delete tombstones and compaction windows must be handled correctly. It is not a substitute for keeping a complete audit history.
- To reconstruct downstream **history**, append immutable change records to a governed store; to reconstruct **current state**, replay ordered

upserts/deletes or take a fresh snapshot.

CHAPTER

## 8. Failure modes and recovery strategy

| Failure | Expected behavior | Recommended response |
|---|---|---|
| Debezium worker crashes | Another worker can resume connector task; duplicates possible | Preserve offsets and slot, restart, validate downstream lag |
| PostgreSQL source unavailable | Streaming pauses; sinks become stale | Preserve authoritative DB recovery, reconnect, monitor WAL/offset continuity |
| PostgreSQL primary failover | Slot continuity depends on PG version, configuration, and managed service | Validate failover-slot configuration before production; never assume offset continuity |
| Kafka broker/cluster outage | Source connector eventually stalls; WAL may accumulate | Restore Kafka, monitor connector and replication-slot disk budget |
| Sink database unavailable | Sink lags while other consumers continue | Retry with backoff; resume from durable offsets; isolate failed consumer |
| Malformed/incompatible schema | Consumer may stop or reject event | Contract checks, quarantine/DLQ, alert, fix and replay; avoid silent skip |
| Duplicate event | Replayed event may arrive again | Idempotent upsert/delete or business ID dedupe |
| Consumer backlog beyond retention | Kafka can no longer replay missing offsets | Restore from archive or perform controlled re-snapshot |
| Unbounded WAL growth | Source disk pressure and risk to transaction availability | Alert on retained WAL/slot lag; throttle source work or repair connector immediately |
| Missing delete or wrong key | Stale rows in read model | Explicit delete handling, stable key design, source-sink reconciliation |

**Subtle failover risk:** In PostgreSQL 17+ there are supported failover logical slots, but the feature must be configured and verified. Older versions and some topologies need different procedures. Always validate the actual database service's documented behavior.

CHAPTER

## 9. Security and compliance design

1. **Data minimization:** allowlist tables and columns. In financial-services examples, do not send PAN, CVV, secrets, or unnecessary customer identifiers into general Kafka topics. Secure or tokenize sensitive values before publication wherever the architecture permits.
2. **Source privilege:** create dedicated replication credentials with only required replication and table permissions; store credentials in a secrets manager and rotate them safely.
3. **Network isolation:** private connectivity among source, connectors, Kafka, registry, and sinks; TLS for transport.
4. **Kafka access control:** topic-level producer/consumer permissions, encrypted storage, separate credentials per sink, and audit logs.
5. **Governance:** classify topics, set retention/deletion rules, track lineage, record approved consumers, and restrict replay access.
6. **Deletion/privacy:** distinguish source row deletes, log compaction, and deletion from archives/backups. Define a separate privacy workflow if legal requirements call for downstream erasure.

**Important:** Merely replicating cardholder data into Kafka can expand regulated-data scope. Prefer excluding it; if unavoidable, design compliance controls with the responsible security team rather than assuming encryption alone is sufficient.

CHAPTER

## 10. Monitoring, SLOs, and operational readiness

| Measurement | Why it matters |
|---|---|
| End-to-end freshness (source commit → sink visible) | Business-facing proof of timeliness |
| Connector task state and restart count | Detect capture failures and restart storms |
| Source LSN progress and replication-slot lag | Detect pinned WAL before source disk exhaustion |
| PostgreSQL disk usage and retained WAL bytes | Protect the OLTP system |
| Kafka broker throughput and partition health | Ensure durable buffering at peak |
| Per-consumer partition lag and processing age | Detect slow or broken sinks |
| Error rate, retries, DLQ volume, schema rejections | Find bad data and unrecovered processing |
| Source vs. sink record counts / sampled hash comparisons | Find silent data divergence |

**Example alert policy (illustrative):** warn when end-to-end delay exceeds the 10-second target persistently; page before WAL retention threatens source capacity; page if connector source progress stops while the source is writing. Thresholds must be based on tested capacity and recovery time, not copied as universal defaults.

**Operational runbooks required:** restart connector without dropping slot; recover failed consumer; replay a topic; backfill after retention loss; migrate schema; handle primary failover; verify reconciliation; rotate secrets.

CHAPTER

## 11. Validation strategy before production

- **Correctness:** compare initial snapshot counts and selected checksums with source; test concurrent updates during snapshot.
- **Change coverage:** exercise create, update, delete, primary-key change, and schema changes.
- **Failure injection:** terminate source connector and sink mid-write, disconnect Kafka, and force a source primary failover.
- **Duplicate/reorder protection:** replay records; verify a latest-state sink does not regress and side effects are not duplicated.
- **Capacity:** test snapshot load, steady-state throughput, 10x spike, consumer catch-up, and WAL growth during outages.
- **Security:** verify topic ACLs, secret handling, TLS, data minimization, and restricted replay access.
- **DR:** confirm recovery point and recovery time from actual tests; a backup that was not restored successfully is not a validated recovery plan.

CHAPTER

## 12. Alternatives and when to select them

| Option | Best fit | Major limitation |
|---|---|---|

| | | |
|---|---|---|
| Timestamp polling | Small, simple, low-risk tables | Deletes and missed writes require extra safeguards; source query load |
| Log-based CDC + Kafka | Many consumers need near-real-time, replayable row changes | More platform components and eventual consistency |
| Database-native logical replication | Building PostgreSQL-to-PostgreSQL replicas | Narrower than a governed multi-consumer event pipeline |
| Direct CDC to managed warehouse | One analytical destination, limited replay needs | Tighter coupling and lower general-purpose fan-out |
| Application dual write (DB + broker) | Generally avoid | Partial failure causes database/event divergence |
| Transactional outbox + CDC | Guaranteed correspondence between business state and a domain-event record | Requires application schema and business-event design |
| Periodic batch ETL | Daily reports, low freshness requirements | High latency; not event-driven |

CHAPTER

# 13. Architecture decision records — concise summary

**ADR-01 — WAL-based CDC instead of polling.** Selected for committed row changes and lower recurring source query load. Rejected polling because deletes, multiple updates, and tracking boundaries are harder to guarantee.

**ADR-02 — Kafka as durable buffer.** Selected for replay and independent consumers; rejected direct synchronous fan-out because slow sinks should not affect the source transaction path.

**ADR-03 — Stable primary-key partitioning.** Selected to preserve per-row order in each topic; rejected random partitioning because updates to one record could be consumed out of order.

**ADR-04 — At-least-once with idempotent sinks.** Selected because it is robust across restart and replay. Rejected claiming universal end-to-end exactly-once behavior across arbitrary external databases and side effects.

**ADR-05 — Governed schemas and least-privilege access.** Selected to prevent breaking consumers and uncontrolled spread of sensitive source records; rejected unversioned free-form payloads.

**ADR-06 — Snapshot + stream with recovery runbooks.** Selected for a complete initial state plus later changes; rejected naïve bulk copy plus unrelated stream start because there can be a gap.

CHAPTER

# 14. Interview-ready explanation (one-minute narrative)

> "I would keep PostgreSQL as the source of truth and use log-based CDC instead of polling to observe committed row changes. A Debezium PostgreSQL connector in Kafka Connect would perform the initial snapshot and then stream WAL changes into Kafka. I would partition by a stable record ID to preserve per-key order and give each downstream application its own consumer group. The key decisions are at-least-once processing with idempotent sinks, versioned schemas, WAL/connector/consumer-lag monitoring, and careful handling of deletes and source failover. Analytics and search can lag independently without affecting payment transactions. If consumers need meaningful business events, I would add a transactional outbox rather than treat every row update as a business event."

### What makes this a strong Solution Architect design

- It starts from **source correctness and business boundaries**, not a technology shopping list.

- It distinguishes **database row changes from domain events**.
- It covers the operational trap: **replication slots can retain WAL** and threaten the source database.
- It states precisely where **ordering and delivery promises stop**.
- It protects sensitive information and gives each consumer an independent recovery path.
- It identifies the trade-off between **replayability and added platform complexity**.

---

CHAPTER

## 15. Official technical references

These are vendor/project-maintained sources for the chosen example stack. Specific connector properties and behaviors depend on deployed versions; check the actual version before implementation.

1. **Debezium — PostgreSQL connector:** https://debezium.io/documentation/reference/stable/connectors/postgresql.html Snapshot, `c/u/d/r` event structure, connector task behavior, failure recovery, PostgreSQL failover, WAL management.
2. **Debezium — Architecture:** https://debezium.io/documentation/reference/stable/architecture.html Kafka Connect topology and connector-to-consumer flow.
3. **PostgreSQL — Logical decoding concepts:** https://www.postgresql.org/docs/current/logicaldecoding-explanation.html WAL, replication slots, replay, restart/duplicate behavior, retained WAL risk.
4. **Apache Kafka — Design:** https://kafka.apache.org/42/design/design/ Partitions, consumer groups, and messaging semantics.
5. **Apache Kafka — Kafka Connect user guide:** https://kafka.apache.org/42/kafka-connect/user-guide/ Connect worker/runtime operations and connector management.
6. **Confluent — Schema evolution and compatibility:** https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html Compatibility modes and schema governance. (Product-specific registry implementation.)
7. **Debezium — Outbox event router:** https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html Transactional outbox pattern for reliable domain-event publication.
8. **PostgreSQL — Replication configuration:** https://www.postgresql.org/docs/current/logical-replication-config.html Logical replication server configuration and sizing controls.

---

**Key takeaway:** A successful CDC pipeline is not simply “Postgres → Debezium → Kafka.” Its design must make snapshot correctness, repeatable application of changes, ordered keys, WAL safety, replay, privacy, and source failover explicit.